

Evaluation Integrity as an Engineering Concern: Receipts, Split Isolation and Rehearsal for AI-Enabled Testing Systems

Abstract—An AI-enabled testing system is only as trustworthy as the machinery that measures it, and that machinery is software with its own defects. We report the engineering of an evaluation harness for an LLM-based mutation test generator, and what it cost when one control was missing. Frozen hash-bound receipts, structural split isolation and retained raw outputs let us establish that a 9.04 point gain on a checkpoint-selection panel corresponded to +3.4346 points on a locked split at exact McNemar $p = 0.0783$, failing a predeclared $p < 0.01$ and retaining the untrained base model — and that candidate reference validity regressed 11.8726 points over the same comparison while parse and execution validity did not. One control was absent: no stage of the one-time final measurement path had ever been executed against real data. That measurement was authorized and consumed without producing anything, because a record-admission rule ran before the scope filter that the production evaluator applied first. We introduce a receipt-bound, full-scale rehearsal on a permitted split as a standing pre-measurement gate; it reproduced a previously recorded value exactly, establishing execution fidelity without touching the consumed split. We argue that such gates are an AI-engineering deliverable, not incidental tooling.

Index Terms—AI engineering, evaluation integrity, MLOps, provenance, software testing, quality assurance

I. INTRODUCTION

Systems with learned components are evaluated by systems without them, and the second kind is ordinary software: it has defects, it drifts, and it is rarely tested as carefully as the model it judges. The AI-engineering literature has documented the operational burden of ML systems [1], [2] and the reproducibility problems that follow [3], [4]; leakage is a recognised cause of overstated results [5]. Less attention is paid to the harness as a build artifact with its own quality requirements.

This short paper reports that engineering problem concretely. It is not a paper about a new model. The model here is a small open-weights code LLM used unchanged, and the contribution is the measurement infrastructure around it: what it caught, what it failed to catch, and what the failure cost.

II. THE SYSTEM AND ITS CONTROLS

The system generates unit tests intended to distinguish a mutated function from its correct implementation, scored by Kill@k — the fraction of targets for which one of the first k candidates kills the mutant. Fig. 1 shows the evaluation architecture and where each integrity control sits.

TABLE I
HEADLINE MEASUREMENT. THE PANELS ARE NOT COMPARABLE QUANTITIES.

Panel	Arm	Kill@8	Functions
Development (selection-only, $n=542$)	base	0.605166	328
	A@431	0.695572	377
Locked ($n=757$)	base	0.594452	450
	A@431	0.628798	476

Paired locked difference: +3.4346 points, +114/−88, net +26, exact McNemar $p = 0.0783$ vs. predeclared $p < 0.01$ — **not met**.
Decision: retain the untrained base model.
Reference validity: 0.452114 → 0.333388 (−11.8726 points), while parse (+0.1486) and execution (+1.3870) validity did not regress.

Four controls were in place from the start. **Frozen receipts:** the generation protocol, the evaluation scope and the decision rule are committed to hash-bound JSON artifacts before the run they govern, and every source file that can change a result is bound by an individual digest rather than a whole-tree hash. **Split isolation:** training, checkpoint-selection, locked and final splits have distinct roles enforced in code. **Raw-output retention:** every candidate retains its model output and a digest of it. **Explicit generation semantics:** parse mode, seed, batch size and candidate count are recorded in the receipt and re-verified at run time.

III. WHAT THE CONTROLS BOUGHT

Table I summarises what the controls made reportable. Because the decision rule was frozen in a hashed artifact before either locked arm ran, a p of 0.0783 against a required $p < 0.01$ could not be renegotiated afterwards, and the retention decision is interpretable. Because reference validity was measured alongside Kill@k, a substantial regression became visible that a kill-rate-only harness would have missed — and because parse and execution validity were measured too, that regression could be isolated rather than attributed to general degradation.

We state the evidential status precisely. The locked point estimate is positive; the predeclared threshold was not met; that is insufficient evidence of an effect, **not** evidence of no effect [6], [7]. Development figures are selection statistics, since that panel chose the checkpoints it scored. No final-test result exists, no repository-level kill rate exists, and no comparison against any external baseline tool was run.

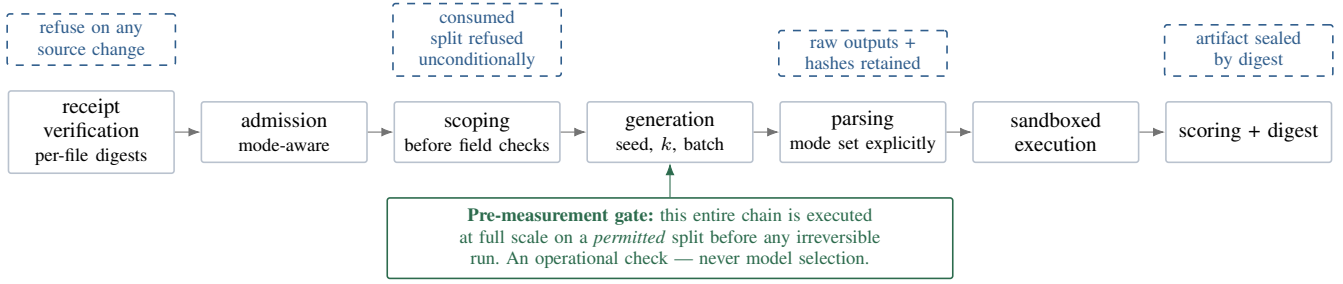


Fig. 1. Evaluation architecture for an AI-enabled testing system. Dashed boxes are integrity controls. The green gate is the control this project lacked, and added only after losing a one-time measurement.

IV. THE CONTROL THAT WAS MISSING

A fourth split was reserved for a single final measurement, protected by a one-time authorization and a hash-bound receipt. It was authorized, attempted, and consumed without producing a measurement: the authorization was spent and the run raised while admitting records, yielding **zero candidates and zero reportable metrics**. The split may never be rerun.

The cause is instructive because it is a pure engineering defect. The final-run path required a fixed set of record fields to be non-empty on *every* record in the split, raising before returning any of them. Records of one execution mode legitimately carry some of those fields blank, and the production evaluator had always loaded the split and *then* scoped scoring by mode. The final path inverted that order.

Two properties made this expensive rather than merely annoying:

- 1) **The component was designed to be unreachable** until the authorization was spent, so it had never been executed against real data. Six successive audits of the readiness receipt each found a real defect — and every one of those defects was in something *recorded* rather than *executed*.
- 2) **The guard asked the wrong question.** It gated on whether access had ever been authorized, a fact that once recorded never expires, rather than on whether anything remained to measure. A permission check is not an exhaustion check.

V. THE REHEARSAL GATE

The repair was not only to fix the admission rule but to remove the condition that made it untestable. Admission now partitions by execution mode first and applies field requirements only to records that will be scored, importing the mode semantics from the production evaluator rather than restating them.

More importantly, we added the gate shown in Fig. 1: before any irreversible measurement, the entire chain executes at full scale on a permitted split under a hash-bound receipt whose source bindings are verified before the model loads. The first such rehearsal scoped 594 requested records to 542 scored targets with zero admission errors, generated 4,336 candidates in 271 batches, retained 4,336 raw outputs with 4,336 digests,

TABLE II
EVALUATION-INTEGRITY RISKS AND THE CONTROL EACH REQUIRES. EVERY MITIGATION LISTED IS IMPLEMENTED IN THE SYSTEM DESCRIBED HERE.

Risk	Implemented control
Decision rule adjusted after seeing results	Rule and thresholds committed to a hash-bound receipt before the run; runtime refuses a receipt whose digest differs
Silent change to code that defines a result	Per-file canonical digests for every result-defining source; whole-tree hashes rejected as too noisy to act on
Same checkout appears to differ across platforms	Both a raw and a line-ending-normalised digest recorded per file
Held-out split reached by ordinary code	Split roles enforced structurally; consumed split refused unconditionally, with the read path deleted rather than made unreachable
Score attributable to a scoring bug, unverifiable later	Raw model output plus digest retained per candidate; digests independently recomputable
Measurement scored under the wrong parser	Parse mode written into the receipt and set explicitly at run time, never inherited from a default
Quality regression hidden by the headline metric	Parse, execution and reference validity reported alongside Kill@k
Irreversible run executes never-run code	Full-scale rehearsal on a permitted split as a standing gate
One failure consumes a one-time resource	Two-phase authorization: reserve on open, commit only after the first batch scores (<i>designed; not yet implemented</i>)

reported zero missing and zero mismatched, and recorded zero mismatches on an independent recomputation. It completed in 577.753 seconds, wrote no weights and loaded no adapter. Its Kill@8 was 0.605166 — identical to the value previously recorded for the same model on the same panel under the same evaluation-scope digest.

That exact reproduction is a statement about code, not about the model. It establishes execution fidelity: the rebuilt path computes the same number as the path that produced the historical value. It is not a model-performance result, not a generalization estimate and not a selection result. The rehearsal receipt carries `final_test_measurement: false`, `eligible_for_model_selection: false` and `supports_performance_claim: false` as machine-checkable fields, and the runner refuses any receipt lacking them — an engineering choice that makes the limitation enforceable rather than editorial.

VI. WHY THIS IS AN AI-ENGINEERING CONTRIBUTION

Three reasons, stated plainly.

First, **the harness is the deliverable that determines whether any model claim is admissible**. Our strongest empirical statements are negative or inconclusive, and they are only reportable because the rule was frozen and the provenance held. Infrastructure that makes a negative result publishable is doing the same job as infrastructure that makes a positive one credible.

Second, **the defects we hit are software defects with software remedies** — ordering of a filter and a validator, a permission check standing in for an exhaustion check, a component unreachable by design and therefore untested. None required ML expertise to cause or to fix, and all of them threatened the validity of an ML result.

Third, **irreversible evaluation resources need release engineering**. A one-time held-out split is a resource that can be spent once and destroyed by a crash in ordinary code. Treating it like a production deployment — rehearse at full scale, verify bindings before the expensive step, make the commit point two-phase — is the discipline the situation calls for, and it is ours to supply.

VII. THREATS TO VALIDITY

One model, one corpus, one supervision family, one seed; we do not claim the measured magnitudes transfer. Kill@k credits any distinguishing candidate, including a universally failing test, which is why reference validity is reported beside it; mutants are an imperfect fault proxy [8]. The locked comparison is a single paired exact test on 202 discordant pairs. One locked criterion failed through a defect of ours — an adapter-specific digest placed inside a contract required to be identical across arms — recorded as a failure rather than reinterpreted; the decision does not rest on it. The two-phase authorization in Table II is designed but not yet implemented, and is labelled as such.

VIII. CONCLUSION

Evaluation infrastructure for AI-enabled systems is software, and it fails in software ways with scientific consequences. Frozen receipts, per-file source binding, structural split isolation and retained raw outputs let us report a locked result that did not meet its predeclared threshold, and a validity regression that the headline metric concealed. Their absence in one place — a measurement path that had never been executed against real data — consumed a one-time resource and produced nothing. The gate that closes that hole is cheap and repeatable: run the whole path, at full scale, on data you are allowed to spend.

An anonymized replication package contains the frozen protocol, decision-rule and result receipts, the rehearsal receipts, and per-figure data files carrying their source digests. No material from the consumed split is included, and none exists.

REFERENCES

- [1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
- [2] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, “Software engineering for machine learning: A case study,” in *IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 291–300.
- [3] O. E. Gundersen and S. Kjenmo, “State of the art: Reproducibility in artificial intelligence,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
- [4] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, and H. Larochelle, “Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program,” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-303.html>
- [5] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023.
- [6] R. L. Wasserstein and N. A. Lazar, “The ASA statement on p-values: Context, process, and purpose,” *The American Statistician*, vol. 70, no. 2, pp. 129–133, 2016.
- [7] V. Amrhein, S. Greenland, and B. McShane, “Scientists rise up against statistical significance,” *Nature*, vol. 567, no. 7748, pp. 305–307, 2019.
- [8] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, “Are mutants a valid substitute for real faults in software testing?” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 654–665.